Estructuras de Datos

Parcial – Planilla dispersa

Una **planilla** es una matriz de enteros de `F` filas y `C` columnas. En la práctica, la gran mayoría de sus celdas valen `0`, por lo que guardar la matriz completa desperdiciaría mucha memoria. Por eso se usa una representación **dispersa**: solo se guardan las celdas cuyo valor es **distinto de 0**, y cualquier celda que no esté guardada vale 0. El tipo abstracto `Planilla` tiene, entre otras, las siguientes operaciones (K es la cantidad de celdas guardadas en la fila involucrada):

- **`Planilla nuevaPlanilla(int f, int c)`**, que crea una planilla de `f` filas y `c` columnas con todas sus celdas en 0. O(F).
- **`int filas(Planilla p)`** e **`int columnas(Planilla p)`**, que indican las dimensiones de la planilla. O(1).
- **`int valor(int f, int c, Planilla p)`**, que indica el valor de la celda `(f, c)`. *Precondición:* la celda existe en la planilla. O(K).
- **`void asignar(int f, int c, int v, Planilla p)`**, que hace que la celda `(f, c)` pase a valer `v`. Si `v` es 0, la celda deja de estar guardada. *Precondición:* la celda existe en la planilla. O(K).
- **`void sumarFila(int origen, int destino, Planilla p)`**, que le suma a cada celda de la fila `destino` el valor de la celda de la misma columna en la fila `origen`. La fila `origen` no se modifica. *Precondición:* ambas filas existen y son distintas. O(K_{origen} + $K_{destino}$).
- **`int cantNoNulos(Planilla p)`**, que indica cuántas celdas distintas de 0 tiene la planilla. O(1).

Para implementar el TAD se utilizará la siguiente representación:

```
struct CeldaNode {
    int col;            // columna de la celda
    int val;            // valor de la celda
    CeldaNode* sig;     // siguiente celda guardada de la MISMA fila
};

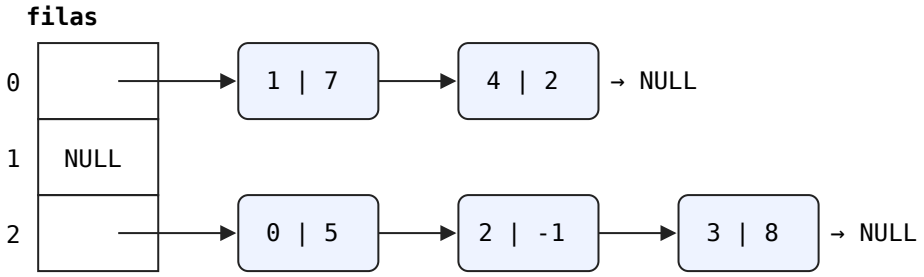
struct PlanillaStr {
    int F;              // cantidad de filas
    int C;              // cantidad de columnas
    CeldaNode** filas;  // array de F punteros: lista de celdas de cada fila
    int noNulos;        // cantidad total de celdas guardadas
};

typedef PlanillaStr* Planilla;
```

Cada fila `i` se representa con una lista simplemente enlazada que empieza en `filas[i]` y que guarda sus celdas **ordenadas por columna de menor a mayor**. El array `filas` se pide con `new CeldaNode*[F]`. Por ejemplo, la siguiente planilla de 3 × 5:

	0	1	2	3	4
0	0	7	0	0	2
1	0	0	0	0	0
2	5	0	-1	8	0

se representa así (cada nodo muestra `col | val`):



Se proveen implementadas las siguientes funciones:

```
Planilla nuevaPlanilla(int f, int c) {
    Planilla p = new PlanillaStr;
    p->F = f;
    p->C = c;
    p->noNulos = 0;
    p->filas = new CeldaNode*[f];
    for (int i = 0; i < f; i++) {
        p->filas[i] = NULL;
    }
    return p;
}

int filas(Planilla p)       { return p->F; }
int columnas(Planilla p)    { return p->C; }
int cantNoNulos(Planilla p) { return p->noNulos; }
```

Ejercicios

> **Importante:** en todos los ejercicios se evalúa el uso eficiente de recursos y la justificación de los costos. No se permite usar recursión.

1. Escribir los **invariantes de representación** de `PlanillaStr`.
2. Implementar, como implementador del TAD, la operación **`valor`** en O(K). Aprovechar que las celdas están ordenadas para cortar la búsqueda lo antes posible.

3. Implementar **`asignar`** en O(K). Hay tres situaciones posibles: la celda ya está guardada y hay que cambiar su valor, la celda ya está guardada y hay que borrarla (porque `v` es 0), o la celda no está guardada y hay que insertarla **en su lugar ordenado**. Se debe recorrer la fila con un **puntero a puntero** (`CeldaNode** pp`) de forma que ni la inserción ni el borrado traten aparte el caso del primer nodo de la fila.

   > **AYUDA:** si `pp` queda apuntando a la flecha donde "debería" estar la celda, insertar un nodo nuevo ahí es tan simple como hacer que el nuevo apunte a `*pp` y después hacer `*pp = nuevo`.

4. Implementar **`sumarFila`** en O(K_{origen} + $K_{destino}$): recorriendo **una sola vez** cada fila, como en el merge de dos listas ordenadas. Tener en cuenta que una celda de `destino` puede quedar en 0 después de la suma, y en ese caso debe dejar de estar guardada. No usar `asignar` (¿por qué no sería eficiente?).

5. Implementar como **usuario** del TAD la función **`int traza(Planilla p)`**, que devuelve la suma de las celdas de la diagonal: `(0,0) + (1,1) + (2,2) + ...`, hasta donde alcancen las filas y las columnas. Indicar su costo en el peor caso.

6. Implementar **`void liberarPlanilla(Planilla p)`**, que libera toda la memoria usada por la planilla.

7. Responder justificando: ¿cuándo conviene esta representación y cuándo convendría una matriz común `int**` de F × C? Comparar la memoria usada y el costo de `valor` y `asignar` en ambas.